PRIMEROS PASOS CON
FASTAPI
ANDRÉS CRUZ YORIS
DESARROLLADOR

Primeros pasos con FastApi

Aquí continúa tu camino en el desarrollo de aplicaciones web en Python con FastApi

Andrés Cruz Yoris

Esta versión se publicó: 2023-08-03

¡Postea sobre el libro!

Por favor ayuda a promocionar este libro.
El post sugerido para este libro es:
¡Acabo de comprar el libro "Primeros pasos con FastApi" de @LibreDesarrollo!

Hazte con tu copia en:

https://www.desarrollolibre.net/libros/libro-primeros-pasos-con-fastapi

Sobre el autor

Este libro fue elaborado por Andrés Cruz Yoris, Licenciado en Computación, con más de 10 años de experiencia en el desarrollo de aplicaciones web en general; trabajo con PHP, Python y tecnologías del lado del cliente como HTML, JavaScript, CSS, Vue entre otras y del lado del servidor como Laravel, Flask, Django y CodeIgniter. También soy desarrollador en Android Studio, xCode y Flutter para la creación de aplicaciones nativas para Android e IOS.

Pongo a tú disposición parte de mi aprendizaje, reflejado en cada una de las palabras que componen este libro, mi doceavo libro en el desarrollo de software, pero el segundo enfocado a Python, para el desarrollo de aplicaciones web con FastApi.

Copyright

Ninguna parte de este libro puede ser reproducido o transmitido de ninguna forma; es decir, de manera electrónica o por fotocopias sin permiso del autor.

Prólogo

FastAPI es un framework web estupendo para la creación de APIs web con Python; nos ofrece múltiple características con las cuales es posible la creación de APIs modulares, bien estructuradas, escalables y con muchas opciones como validaciones, formatos, tipados entre otros.

Al instalar FastAPI, se instalan dos módulos muy importantes:
Pydantic que permite la creación de modelos para la validación de datos.
Starlette, que es un tooltip ASGI liviano, utilizado para crear servicios web asíncronos (o síncronos) en Python.

Con estos paquetes, tenemos lo básico para crear APIs, pero, podemos extender fácilmente un proyecto en FastAPI con otros módulos para dotar a la aplicación de más características, como lo son la base de datos, motores de plantillas entre otros.

FastAPI es un framework fácil de aprender e iniciar y de alto rendimiento; es ideal para crear toda clase de sitios que no solamente consisten en APIs, si no, podemos instalar un manejador de plantillas para devolver páginas web completas.

Este libro es mayoritariamente práctico, iremos conociendo los fundamentos de FastAPI, conociendo sus características principales en base a una pequeña aplicación que iremos expandiendo capitulo tras capitulo.

Para quién es este libro

Este libro está dirigido a cualquier persona que quiera aprender a desarrollar en FastApi sus primeras APIs.

Para aquellas personas que conozcan cómo programar en Python u otros frameworks web.

Para aquellas personas que quieran aprender algo nuevo.

Para las personas que quieran mejorar una habilidad y que quieran crecer como desarrollador y que quiera seguir escalando su camino en el desarrollo de aplicaciones con Python.

Consideraciones

Recuerda que cuando veas el símbolo de $ es para indicar comandos en la terminal; este símbolo no lo tienes que escribir en tu terminal, es una convención que ayuda a saber que estás ejecutando un comando.

Al final de cada capítulo, tienes el enlace al código fuente para que lo compares con tu código.

El uso de las negritas en los párrafos tiene dos funciones:
Si colocamos una letra en negritas es para resaltar algún código como nombre de variables, tablas o similares.
Resaltar partes o ideas importantes.

Para mostrar tips usamos el siguiente diseño:

Tips importantes

Para los fragmentos de código:

```
from fastapi import ***, Query
@app.get("/page")
def page(page: int = Query(1, ge=1, le=20), size: int = Query(5, ge=5, le=20)):
    return {"page": page, "size": size}
```

Al emplear los ***

Significa que estamos indicando que en el código existen fragmentos que presentamos anteriormente.

Como recomendación, emplea Visual Studio Code como editor, ya que, es un editor excelente, con muchas opciones de personalización, extensiones, intuitivo, ligero y que puedes desarrollar en un montón de plataformas, tecnologías, frameworks y lenguajes de programación; así que, en general, Visual Studio Code será un gran compañero para ti.

https://code.visualstudio.com/

Este libro tiene un enfoque práctico, por lo tanto, iremos presentando los componentes principales de FastApi, que va desde un hola mundo hasta conocer estructuras más complejas y de esta manera tener un enfoque claro de esta tecnología. Recuerda que puedes consultar desde el índice del libro, los distintos componentes en caso de que quieras revisar algún tema en particular en cualquier momento.

Se recomienda que se lea cada uno de los capítulos en el orden en el cual se encuentran dispuestos ya que, la aplicación que vamos construyendo va evolucionando en cada capítulo y la misma es referenciada en cada uno de los capítulos.

Fe de errata y comentarios

Si tienes alguna duda, recomendación o encontraste algún error, puedes hacerla saber en el siguiente enlace:

https://www.desarrollolibre.net/contacto/create

Por mi correo electrónico:

desarrollolibre.net@gmail.com

O por Discord en el canal de FastApi:

https://discord.gg/sg85Zgwz96

Como recomendación, antes de reportar un posible problema, verifica la versión a la que te refieres y lo agregas en tu comentario; la misma se encuentra en la segunda página del libro.

Introducción

Esta guía tiene la finalidad de dar los primeros pasos con FastAPI empleando Python; con esto, vamos a plantear dos cosas:
No es un libro que tenga por objetivo conocer al 100% FastApi, o de cero a experto, ya que, sería un objetivo demasiado grande para el alcance de esta guía, si no, conocer que nos ofrece, crear los primeros métodos que conforman nuestra API y cómo personalizar las mismas a nivel de argumentos, respuestas entre otros; además de conocer cómo integrar FastAPI con otros componentes de terceros.
Se da por hecho de que el lector tiene conocimientos al menos básicos en el desarrollo de Python.

Este libro tiene un enfoque práctico, conociendo los aspectos claves de la tecnología y pasando a la práctica, implementando de a poco pequeñas características de una aplicación que tiene alcance real.

Para seguir este libro necesitas tener una computadora con Windows, Linux o MacOS.

El libro se encuentra actualmente en desarrollo.

Mapa

Capítulo 1: En este capítulo vamos a presentar el software necesario para crear proyectos en FastAPI, qué pasa desde Python, hasta preparar el entorno.

Capítulo 2: Presentamos algunos comandos imprescindibles para desarrollar en FastApi, prepararemos el entorno y daremos una introducción al framework.

Capítulo 3: Uno de los factores primordiales en FastApi, es la creación de recursos para la API mediante funciones, en este apartado trataremos las bases de esto introduciendo el ruteo entre múltiples archivos, así como las distintas opciones para los argumentos y parámetros de estas rutas.

Capítulo 4: En este apartado aprender a manejar los códigos de estado en HTTP de los métodos de la API y también controlar los errores/excepciones desde los métodos de la API.

Capítulo 5: En este apartado veremos cómo crear datos de ejemplos para utilizar desde la documentación automática que ofrece FastAPI para cada uno de los métodos de la API.

Capítulo 6: En este capítulo veremos cómo implementar la carga de archivos, conociendo las diferentes variantes existentes en FastAPI.

Capítulo 7: En este capítulo veremos cómo conectar una aplicación en FastAPI a una base de datos relacional como lo es MySQL.

Capítulo 8: En este capítulo veremos instalar y emplear un motor de plantillas en Python, específicamente el de Jinja con el cual podremos devolver respuestas en formato HTML.

Capítulo 9: En este capítulo aprenderemos cómo usar las dependencias.

Capítulo 10: En este capítulo veremos cómo usar los middlewares para interceptar peticiones a los métodos de la API y ejecutar algún procedimiento antes del request o después de generar la respuesta.

Capítulo 11: En este capítulo veremos cómo crear un módulo de usuario, para registrar usuarios, realizar el login, generar tokens de acceso y hacer el logout.

Capítulo 12: En este capítulo conoceremos algunas particularidades y funcionalidades más de FastAPI como el uso de las anotaciones y el operador de Ellipsis.

Capítulo 13: En este capítulo veremos cómo implementar pruebas unitarias.

Capítulo 14: En este capítulo conoceremos algunos aspectos generales aplicados a FastAPI.

Indice

- **Capítulo 1: Software necesario e instalación de las herramientas**
  - Software necesario
    - Python
    - Visual Studio Code
    - Navegador web

- **Capítulo 2: Introducción a FastApi**
  - Verificar el comando de acceso a Python 3 y Pip
  - Comandos básicos
  - Preparar el entorno
    - Ambiente virtual
  - Instalar dependencias y prepara el proyecto
    - FastApi
    - Uvicorn
  - Hola Mundo
  - Parámetros en las rutas (path)
  - Tipos de rutas
  - Bibliotecas principales de FastApi

- **Capítulo 3: Ruteo en FastApi**
  - Ruteo entre múltiples archivos
  - Limitar los valores permitidos (datos predefinidos)
  - Parámetros con Query()
  - Path con Path()
  - Body
  - Validación de cuerpos de solicitud utilizando modelos Pydantic
    - Validaciones
    - Anidar clases
    - Herencia
    - Validación por campos
    - Valores opcionales y valores por defecto
    - Tipos de campos

    - Usar una lista como argumento
  - FastAPI documentación automática
    - Swagger
    - ReDoc
  - Conclusiones

- **Capítulo 4: Códigos de estado**
  - Códigos de estado en HTTP
  - Personalizar el código de estado de la respuesta en los métodos
  - Manejo de errores
    - ¿Qué es una excepción HTTP?
    - Evaluar que el índice no está fuera de rango
    - Evaluar que la tarea no haya sido registrada
  - Conclusión

- **Capítulo 5: Declarar datos de ejemplo**
  - Datos de ejemplos en modelos
    - Relaciones anidadas
    - Datos de ejemplos desde el argumento
  - Datos de ejemplos desde el argumento
    - Ejemplo simple
    - Listado de ejemplos

- **Capítulo 6: Upload de archivos**
  - File
  - UploadFile
    - Guardar archivo
    - Manejar múltiples archivos

- **Capítulo 7: Base de datos**
  - Instalar dependencias
  - Configurar conexión
  - Cambios en los modelos
  - Crear modelos (tablas) de la base de datos
  - Utilizar la conexión y crear las tablas en la base de datos
  - Inyectar dependencia de la sesión de la base de datos en las rutas
    - ¿Para qué se emplea la clase Depends?
  - Demo: yield vs return
  - Métodos CRUD para las tareas, base de datos

- Paginación
- Relaciones
  - Relación de tipo One to Many
  - Relación de tipo Many to Many
  - Cambios en el CRUD de tareas (métodos de la API)
- Remover columnas desde las consultas en SQLAlchemy
- Fechas

- **Capítulo 8: Template en FastAPI**
  - Sobre Jinja
  - Primeros pasos con Jinja
    - Bloques de control
    - Filtros
    - Establecer variables
    - Bloques
    - Herencia de templates
  - Uso de Jinja en FastAPI
    - Instalar Jinja
    - Listado en HTML
    - Peticiones en JavaScript
    - Formularios en HTML

- **Capítulo 9: Dependencias**
  - Dependencias en el path (decorador de los métodos de la API)
  - Definir dependencias como variables

- **Capítulo 10: Introducción a los Middleware**
  - Crear un middleware

- **Capítulo 11: Autenticación**
  - Autenticación básica
  - Autenticación por base de datos
  - Métodos para la API
    - Proteger rutas con usuario autenticado
    - Protección de los métodos de la API mediante la dependencia OAuth2PasswordBearer
  - Verificar token de acceso al momento de crear el token

- **Capítulo 12: Anotaciones, notación de Ellipsis (...) y tipos devueltos**

    - Notación de Ellipsis
    - Anotaciones/Annotated
        - Ejemplos usando Annotated
    - Tipos devueltos

- **Capítulo 13: Testing/Pruebas en aplicaciones de FastAPI**
    - pytest, para las pruebas unitarias
        - Creando las primeras pruebas unitarias
        - Eliminando la repetición con pytest fixtures
        - Extra: Loop event
        - Configuraciones bases
        - Probando el módulo de usuario
        - Probando el módulo de tareas
    - Usando testClient de FastAPI
        - Configuración base
        - Probando el módulo de usuario y tarea
        - Otras pruebas: Definir módulos
    - Conclusiones

- **Capítulo 14: Aspectos generales**
    - VSC: Seleccionar intérprete
        - Modo debug en VSC
    - Tareas en segundo plano

Comunitaria

Esta es la versión Comunitaria y es completamente GRATUITA, puedes obtener el libro completo desde: **desarrollolibre.net/libros**.

Este recurso es TOTALMENTE GRATUITO y NO debes de pagar por el bajo ningún concepto y es mi pequeño agradecimiento "A la comunidad de Internet", de la cual, he aprendido tanto sin gastar un solo dolar.

El modo oscuro de este PDF esta disponible SOLO en **https://academy.desarrollolibre.net/**

Capítulo 1: Software necesario e instalación de las herramientas

Python es un lenguaje excelente para empezar a programar, es sencillo, modular, y por su misma estructura al emplear la indentación para delimitar la estructura, el código queda correctamente indentado; con conocimientos básicos en Python, podemos usarlos para dominar en buena medida parte del framework como vamos a conocer en este libro.

Software necesario

Como software fundamental, necesitamos Python, este es el único programa que tienes que instalar de manera obligatorio en tu equipo, ya que, es el lenguaje de programación en el cual se encuentra programado FastAPI, y el que tenemos que usar para crear nuestras aplicaciones con este framework; la web de Python viene siendo esta:

https://www.python.org/

Python

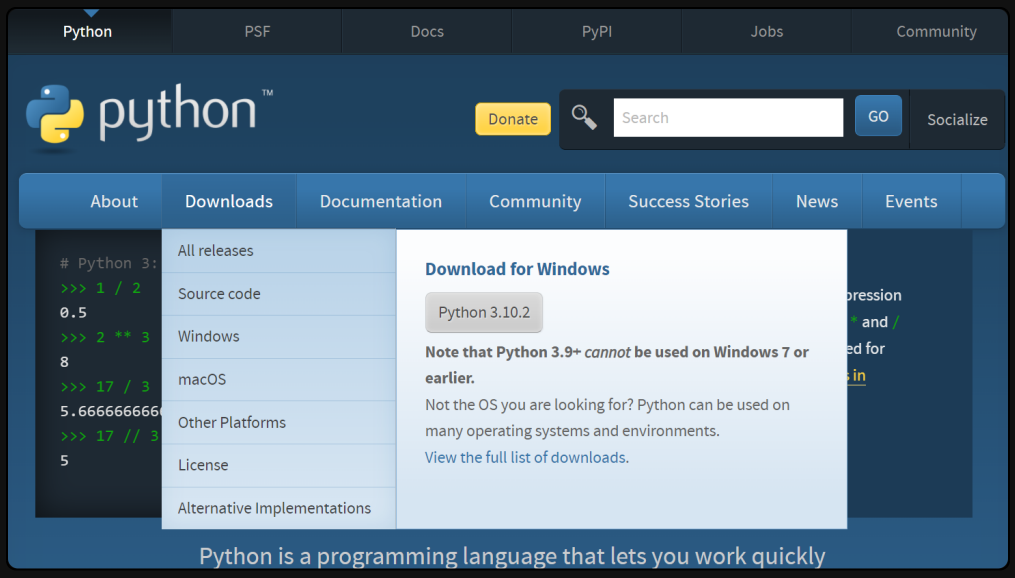
Antes de seguir, vamos a hablar un poco más sobre Python; Python es un lenguaje de programación de alto nivel e interpretado, orientado a objetos y con una sintaxis muy limpia, amigable, fácil de aprender y gran legibilidad con una gran cantidad de paquetes, módulos, librerías y frameworks a nuestra disposición lo hacen muy atractivo para el desarrollo rápido de aplicaciones.

A menudo, los programadores se enamoran de Python debido a la mayor productividad que proporciona. Dado que no hay un paso de compilación, el ciclo de edición, prueba y depuración es increíblemente rápido y fácil.

No hay necesidad de que instales una nueva versión de Python ya que, puedes trabajar con la que ya incorpora tu sistema.

Pero, en caso de que necesites instalar Python, es extremadamente sencillo; vamos a la web oficial, en el apartado de descarga:

https://www.python.org/



Lo descargamos, ejecutamos y es el típico "next next next" para instalar el mismo; muy importante que agregues Python al PATH de Windows:

Visual Studio Code

Como editor de código, vamos a usar Visual Studio Code ya que, es un editor, excelente, con muchas opciones de personalización, extensiones, intuitivo, ligero y que puedes desarrollar en un montón de plataformas, tecnologías, frameworks y lenguajes de programación; así que, en general, Visual Studio Code será un gran compañero para ti; pero, si prefieres otros editores como Sublime Text, o similares, puedes usarlo sin ningún problema.

https://code.visualstudio.com/

Extensión

Si usas VSC, tienes que instalar la extensión de Python:

https://marketplace.visualstudio.com/items?itemName=ms-python.python

Navegador web

Como navegador web, te recomiendo Google Chrome; aunque es cierto que al momento de desarrollar en tecnologías web, es recomendable emplear más de un navegador; al desarrollar específicamente para el lado del servidor y no enfocarnos en desarrollar del lado del cliente; no tiene

mucho sentido para este libro, que emplees múltiples navegadores; dicho esto, puedes emplear cualquier otro navegador en caso de que Google Chrome no sea de tu agrado:

https://www.google.com/intl/es/chrome/

Capítulo 2: Introducción a FastApi

En este primer capítulo vamos a conocer algunos comandos imprescindibles para poder trabajar en un proyecto con FatApi, al igual que preparar el entorno de desarrollo mediante los ambientes virtuales y crear el primer ejemplo con FastApi, es decir, el "Hola Mundo".

Verificar el comando de acceso a Python 3 y Pip

Antes de comenzar, es importante que revises cuál versión de Python estás empleando; en algunos sistemas operativos como Linux o MacOS pueden venir instalado una versión antigua de Python, muchas veces la versión 2 de Python con la cual no puedes usar FastApi; para poder crear nuestras aplicaciones con FastApi es necesario que tengas la versión 3.

Una vez instalado Python 3 en tu equipo, verifica cual es la forma de acceder al mismo; en tu terminal, si ejecutas el siguiente comando:

```
$ python -V
```

Y ves algo como esto:

```
command not found: python
```

Significa que Python no ha sido instalado o al menos no es la forma de acceder a Python 3; si ves una salida como esta:

```
Python 3.X
```

Significa que para acceder al intérprete de Python debes de escribir simplemente "python" en tu terminal.

En caso de que el comando anterior no funcionara, puedes probar ejecutar el comando de:

```
$ python3 -V
```

Y deberías de ver una salida como:

```
Python 3.X
```

Donde "X" es una variante de Python 3, como la 3.10.2 lo que significa que esta es la forma de acceder al intérprete de Python en tu PC.

El mismo procedimiento debes de hacer con el instalador de paquetes para Python, el llamado pip:

```
$ pip3 -V
```

O

```
$ pip -V
```

En al menos uno de los casos deberías de ver algo como:

pip 22.X from /Library/Frameworks/Python.framework/Versions/***/pip (python 3.X)

Comandos básicos

Para poder crear nuestras aplicaciones en Python de una manera organizada, al menos debes de conocer los siguientes comandos, que son muy útiles al momento del desarrollo; veamos cuales son.

Para instalar dependencias en Python, que puede ser desde algún complemento para realizar el login social o un framework como Django, Flask o en este caso FastApi, debes de usar la opción de "pip install" seguido del paquete o paquetes que queremos instalar; por ejemplo:

```
$ pip install fastapi
```

Para desinstalar un paquete, tenemos el comando de "pip uninstall"

```
$ pip uninstall fastapi
```

Otro comando muy útil es el que nos permite saber cuáles paquetes tenemos instalado en un proyecto; como sucede con otros ambientes como Node o PHP con Composer, en Python muchas veces cuando los paquetes instalados contienen dependencias; por ejemplo, en el caso de Django, al instalar Django, se instalan las siguientes dependencias:

```
$ pip install django
```

```
asgiref==3.5.0
Django==4.0.3
sqlparse==0.4.2
tzdata==2021.5
```

Para conocer las dependencias, tenemos el siguiente comando:

```
$ pip freeze
```

Cuya salida podemos guardar en un archivo, que por convención se llama cómo requirements.txt:

```
$ pip freeze > requirements.txt
```

Este archivo lo podemos usar ahora para instalar las dependencias de un proyecto con las versiones exactas; muy útil cuando estás desarrollando un proyecto en Python con múltiples personas o cuando pasas a producción o simplemente cuando necesitas ejecutar el proyecto en múltiples PCs:

```
$ pip install -r requirements.txt
```

Preparar el entorno

Ya aclarado algunos comandos básicos que debemos de conocer tanto para ejecutar archivos del proyecto como para mantener el proyecto y sus dependencias, vamos a crear el ambiente virtual para poder crear la primera aplicación en FastApi.

Ambiente virtual

En Python, los ambientes virtuales son entornos de desarrollo aislados al sistema operativo, por lo tanto, los paquetes que instalemos no serán instalados en el sistema operativo si no en el ambiente virtual; podemos crear tantos ambientes virtuales como queramos, usualmente tenemos un ambiente virtual por proyecto y con esto, se manejan de manera independiente todos los paquetes y dependencias necesarios para un proyecto en Python en general.

En Python, los ambientes virtuales se pueden crear y gestionar con la herramienta venv, que viene incluida en la librería estándar de Python aunque también es posible instalarlo mediante:

```
$ pip install virtualenv
```

Este es el único paquete que es necesario tener instalado a nivel global del sistema operativo.

Para crear el ambiente virtual, tenemos el comando de:

```
$ python3 -m venv venv
```

Vamos a ejecutar el siguiente comando:

```
$ python3 -m venv vtasks
```

El cual generará el ambiente virtual, que no es más que una carpeta con varios archivos y carpetas; para activar el mismo, desde la carpeta del ambiente virtual:

```
$ cd vtasks
```

Ejecutamos en MacOS o Linux:

```
$ source vtasks/bin/activate
```

En Windows el comando sería como:

```
$ vtasks\Scripts\activate
```

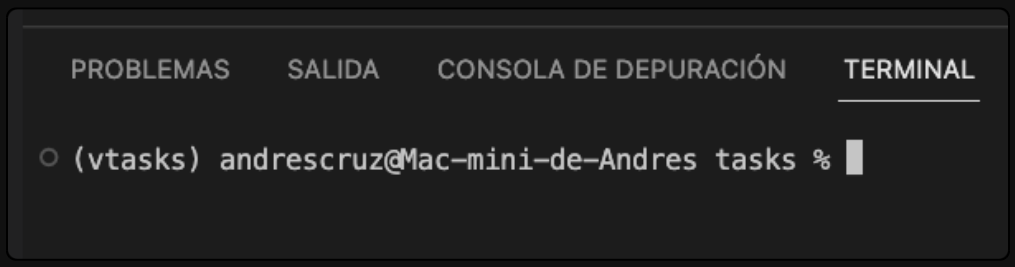
Y con esto, ya tenemos todo listo para indicar nuestro proyecto; recuerda que para ejecutar cualquier comando de Python o de la pip siempre debes de tener activo el ambiente virtual.

Instalar dependencias y prepara el proyecto

Para nuestro primer proyecto, vamos a crear una app de tareas; para ello, dentro de la carpeta del ambiente virtual, crearemos la carpeta del proyecto que llamaremos simplemente tasks; quedando el proyecto de la siguiente manera:

- vtasks
- Include
- Lib
- Scripts -- En Windows
- bin -- En MacOS o Linux
- pyvenv.cfg
- tasks

Esta carpeta llamada "tasks" (no la de vtasks, que corresponde al ambiente virtual) es la que te recomiendo que abras en Visual Studio Code (o el editor de tu preferencia); desde la terminal de VSC recuerda activar el ambiente virtual:



Para poder empezar a trabajar con FastApi, necesitamos instalar dos paquetes que veremos a continuación.

FastApi

FastAPI es un web framework moderno utilizado para la creación de APIs (interfaces de programación de aplicaciones) en Python; para instalar el paquete tenemos:

```
$ pip install fastapi
```

Uvicorn

Unicorn es un servidor web HTTP de tipo ASGI (Asynchronous Server Gateway Interface) que podemos usar en múltiples frameworks webs en Python de tipo asíncronos como lo son Django, Flask y por supuesto FastApi y para instalar el servidor lo podemos hacer como si fuera un paquete más para Python con:

```
$ pip install uvicorn
```

O podemos instalar los dos con un solo comando:

```
$ pip install fastapi uvicorn
```

Recuerda ejecutar el comando anterior con el ambiente virtual activo.

Hola Mundo

Finalmente, es momento de crear el "Hola Mundo" con FastApi; es decir, implementar lo mínimo necesario para poder ver algo por pantalla; así que, creamos un archivo dentro del proyecto (la carpeta tasks):

api.py

Con el siguiente código:

api.py

```
from fastapi import FastAPI
app = FastAPI()
@app.get("/")
def hello_world():
    return {"hello": "world"}
```

En este primer ejemplo, lo primero que hacemos es cargar una clase que proporciona acceso al framework de FastAPI:

from fastapi import FastAPI

Con esta clase, creamos una instancia de FastApi:

```
app = FastAPI()
```

Lo cual nos da acceso a múltiples funcionalidades del framework, como lo es, crear las rutas de la aplicación.

Definimos una petición de tipo GET para la raíz y esto se hace mediante un decorador como el que podemos ver a continuación:

```
@app.get("/")
```

Por supuesto, podemos acceder a otros tipos de peticiones como POST, PUT, PATCH o DELETE indicando el método correspondiente el cual tiene su equivalente directo con el nombre de la petición a usar; es decir, para enviar una petición de tipo GET, usamos la función de get(), para enviar una petición de tipo POST, usamos la función de post().

Al igual que ocurre con otros frameworks web, cada petición es procesada mediante una función, en el ejemplo anterior, la petición de tipo GET para la raíz, es procesada mediante una función llamada hello_world() que lo único que hace es devolver un diccionario indicando el mensaje de "hola mundo":

```
@app.get("/")
def hello_world():
    return {"hello": "world"}
```

Con esto, tenemos nuestro primer ejemplo de hola mundo en FlastApi; pero, para poder ver este mensaje por pantalla, específicamente cualquier programa que permite procesar peticiones HTTP como lo es un navegador, tenemos que levantar el servidor asociado a la aplicación anterior y es aquí en donde usamos el servidor de uvicorn instalado anteriormente; para ello, desde la terminal y raíz del proyecto, usamos el siguiente comando:

```
$ uvicorn api:app --reload
```

Con el comando anterior, indicamos el nombre del archivo, que en este caso es el llamado api.py:

```
api:app
```

Y que el servidor se mantenga atento a los cambios; es decir, con la opción de:

```
--reload
```

El servidor se recargará cada vez que se realicen cambios en la aplicación.

Al ejecutar el comando anterior, veremos por la terminal:

```
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [15820] using StatReload
INFO:     Started server process [4024]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     127.0.0.1:58209 - "GET / HTTP/1.1" 200 OK
INFO:     127.0.0.1:58209 - "GET /favicon.ico HTTP/1.1" 404 Not Found
INFO:     127.0.0.1:58209 - "GET / HTTP/1.1" 200 OK
```

Nos indica cual es la ruta en la cual la aplicación a sido levantada:

http://127.0.0.1:8000

Y desde esta ruta, que podemos ejecutar en el navegador, veremos en mensaje de:

http://127.0.0.1:8000

```
{
  "hello": "world"
}
```

En caso de que sea necesario, también puedes personalizar el puerto mediante la opción de port e indicas el puerto que quieras usar, en este ejemplo, el puerto 8001:

```
$ uvicorn api:app --port 8001
```

Y si quieres colocar también la opción de reload:

```
$ uvicorn api:app --port 8001 --reload
```

Y en la salida verás ahora que el servidor levando la aplicación en el puerto 8001

```
Uvicorn running on http://127.0.0.1:8001 (Press CTRL+C to quit)
```

Finalmente, la función anterior también puedes indicar el tipo de dato de retorno:

```
@app.get("/")
def hello_world() -> dict:
    return {"hello": "world"}
```

Es importante notar que se usa una petición de tipo GET, que es el típico tipo de petición empleada para consultar datos; y la única que podemos emplear directamente desde el navegador sin necesidad de emplear un formulario.

Parámetros en las rutas (path)

A nivel de las rutas también se pueden definir parámetros con los cuales interactúa el usuario final y lucen de la siguiente forma:

```
@app.get("/posts/{id}")
```

Como puedes ver, este esquema de rutas permite emplear parámetros dinámicos y con esto, poder suministrar datos a cada uno de los recursos de la API que estemos construyendo; en el ejemplo anterior, se indica que para la ruta se debe de suministrar un parámetro llamado id que luego es suministrado como un argumento a la función decorada; es decir:

```
@app.get("/posts/{id}")
def add(id: int):
    #TODO
    return {"POST ID": id}
```

En el ejemplo anterior, se debe de indicar un parámetro llamado "id" que debe ser de tipo entero el cual puede ser empleado (por ejemplo) para buscar por el identificador de un post que se quiere recuperar:

/post/123

Tipos de rutas

Existen distintos métodos HTTP que podemos usar para realizar peticiones al servidor; estos métodos no son más que un conjunto de verbos que se utilizan para realizar diferentes tipos de acciones; los métodos más comunes son GET, POST, PUT, PATCH y DELETE:

- GET: se utiliza para obtener información de un servidor web. La información se envía en la URL de la petición.
- POST: se utiliza para enviar información a un servidor web. La información se envía en el cuerpo de la petición.
- PUT: se utiliza para actualizar información en un servidor web. La información se envía en el cuerpo de la petición.
- PATCH: se utiliza para actualizar parcialmente información en un servidor web. La información se envía en el cuerpo de la petición.
- DELETE: se utiliza para eliminar información de un servidor web. La información se envía en la URL de la petición.

Todos estos métodos tienen su equivalente en FastApi; de momento hemos utilizado métodos de tipo GET, para obtener datos, pero, también es posible emplear otros:

```
@app.get(<URI>)
@app.post(<URI>)
@app.put(<URI>)
@app.patch(<URI>)
@app.delete(<URI>)
```

Como puedes ver, tenemos una función decoradora para cada tipo de método y tiene una relación directa con el tipo de petición HTTP; es decir, tenemos un método decorador put() para enviar peticiones de tipo PUT, y así para el resto.

Bibliotecas principales de FastApi

Para terminar esta introducción con FastAPI, es importante señalar que FastAPI se basa en dos bibliotecas principales de Python: Starlette, un framework web ASGI de bajo nivel (**https://www.starlette.io/**), y Pydantic, una biblioteca de validación (**https://pydantic-docs.helpmanual.io/**), del cual esta última biblioteca, la usaremos arduamente en las siguientes

secciones.

Ambas librerías las puedes ver si haces un:

```
$ pip freeze
```

Sobre tu proyecto en FastApi.

Capítulo 3: Ruteo en FastApi

En el capítulo anterior, vimos cómo crear un "hola mundo" en FastApi definimos una ruta de tipo GET que devuelve la respuesta JSON {"hello": "world"}. Para hacer esto, primero creamos una instancia de un objeto FastAPI que importamos antes; esta instancia se guarda en una variable llamada app que es el objeto principal de la aplicación que conectará todas las rutas API; con este objeto, es posible emplear un decorador para definir las rutas. Esta no es la mejor manera que tenemos para trabajar con las rutas, ya que, si tenemos archivos adicionales en las cuales, definen distintas rutas para la aplicación mediante estos decoradores, tendríamos que pasar el objeto app a todos estos archivos; en FastApi también podemos crear una instancia de la clase APIRouter que se puede usar de la misma manera para decorar las funciones (conocidas como funciones de operación) de la ruta como hacíamos con la instancia de FastApi.

Así que, con lo comentado anteriormente, podemos crear el "hola mundo" en FastApi de la siguiente manera:

api.py

```
from fastapi import APIRouter
router = APIRouter()
@router.get("/")
def hello_world():
    return { "hello": "world" }
app.include_router(router)
```

El cambio adicional que tenemos que realizar es la de agregar las rutas definidas antes, para esto, usamos la función de include_router() para incluir la instancia empleada para definir las rutas:

```
app.include_router(router)
```

Recuerda remover o comentar la anterior función para evitar conflictos entre dos rutas iguales; por ejemplo ("/").

Ruteo entre múltiples archivos

Con la clase de API Router que vimos antes, podemos crear aplicaciones realmente modulares, separando en archivos funciones que administran recursos similares; en otras palabras, colocar en

un mismo grupo las funciones que tienen un comportamiento común, como la gestión de tareas, usuarios, etc.

Pensando en lo anterior, vamos a crear un nuevo archivo en el cual, colocaremos el CRUD de las tareas:

task.py

```
from fastapi import APIRouter
task_router = APIRouter()
task_list= []
@task_router.get("/")
def get():
    return { "tasks": task_list }
@task_router.post("/{task}")
def add(task: str):
    task_list.append(task)
    return { "tasks": task_list }
@task_router.put("/")
def update(index: int, task: str):
    task_list[index] = task
    return { "tasks": task_list }
@task_router.delete("/")
def delete(index: int):
    del task_list[index]
    return { "tasks": task_list }
```

Como puedes ver, el código anterior consta de 4 funciones, para crear, actualizar, eliminar y obtener un listado respectivamente; en todos estos ejemplos, se emplea una lista para simular la gestión de tareas usando los métodos anteriormente mencionados.

Para cargar las rutas de este nuevo archivo, debemos de hacerlo desde el archivo principal, que vienen siendo el de api.py; así que:

api.py

```
from fastapi import FastAPI
from fastapi import APIRouter
from task import task_router
app = FastAPI()
router = APIRouter()
@router.get('/hello')
def hello_world():
    return { "hello": "world" }
app.include_router(router)
app.include_router(task_router)
```

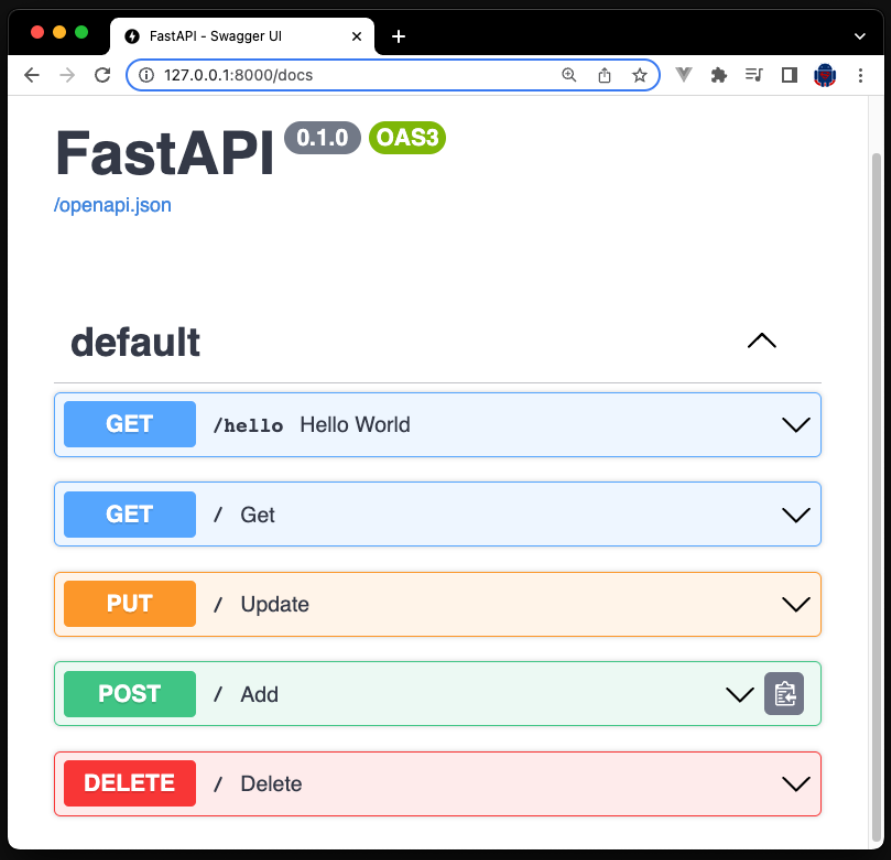
También podemos agregar un prefijo para este set de rutas:

```
app.include_router(todo_router, prefix="/tasks")
```

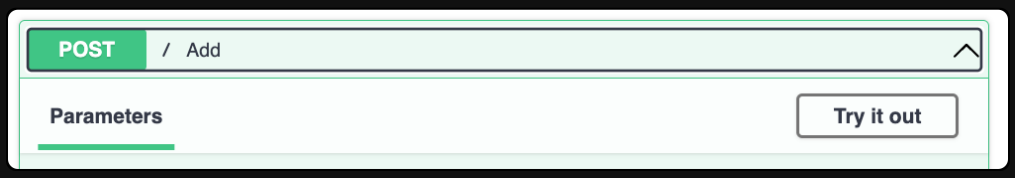
Desde la siguente URL:

http://127.0.0.1:8000/docs

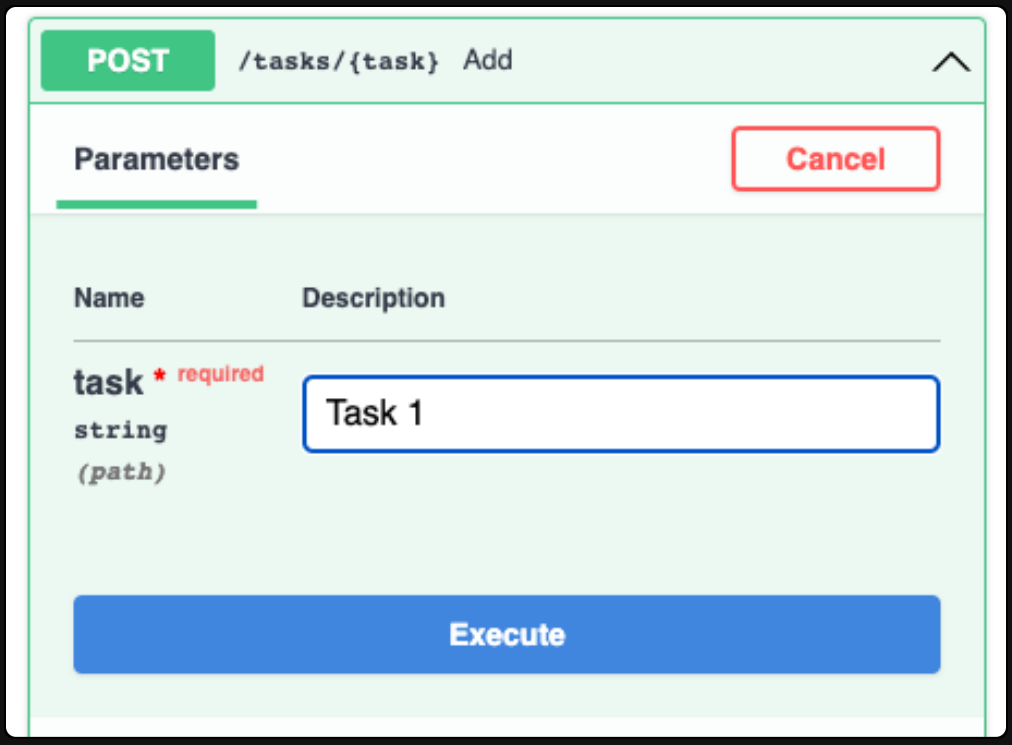
Puedes ver todos los métodos o recursos rest que hayas creado e inclusive probar los métodos desde ese enlace, por lo tanto, no hace falta emplear soluciones como Postman ya que, todo viene integrado en el framework de FastApi; actualmente, deberías de ver los siguientes:



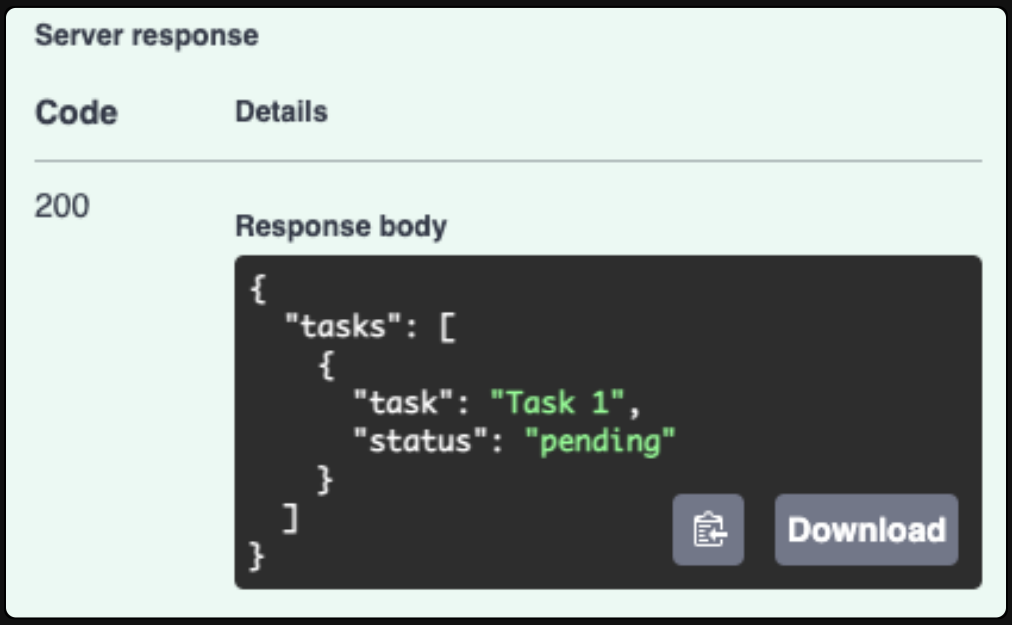
Desde esta sección, puedes probar los recursos anteriores:



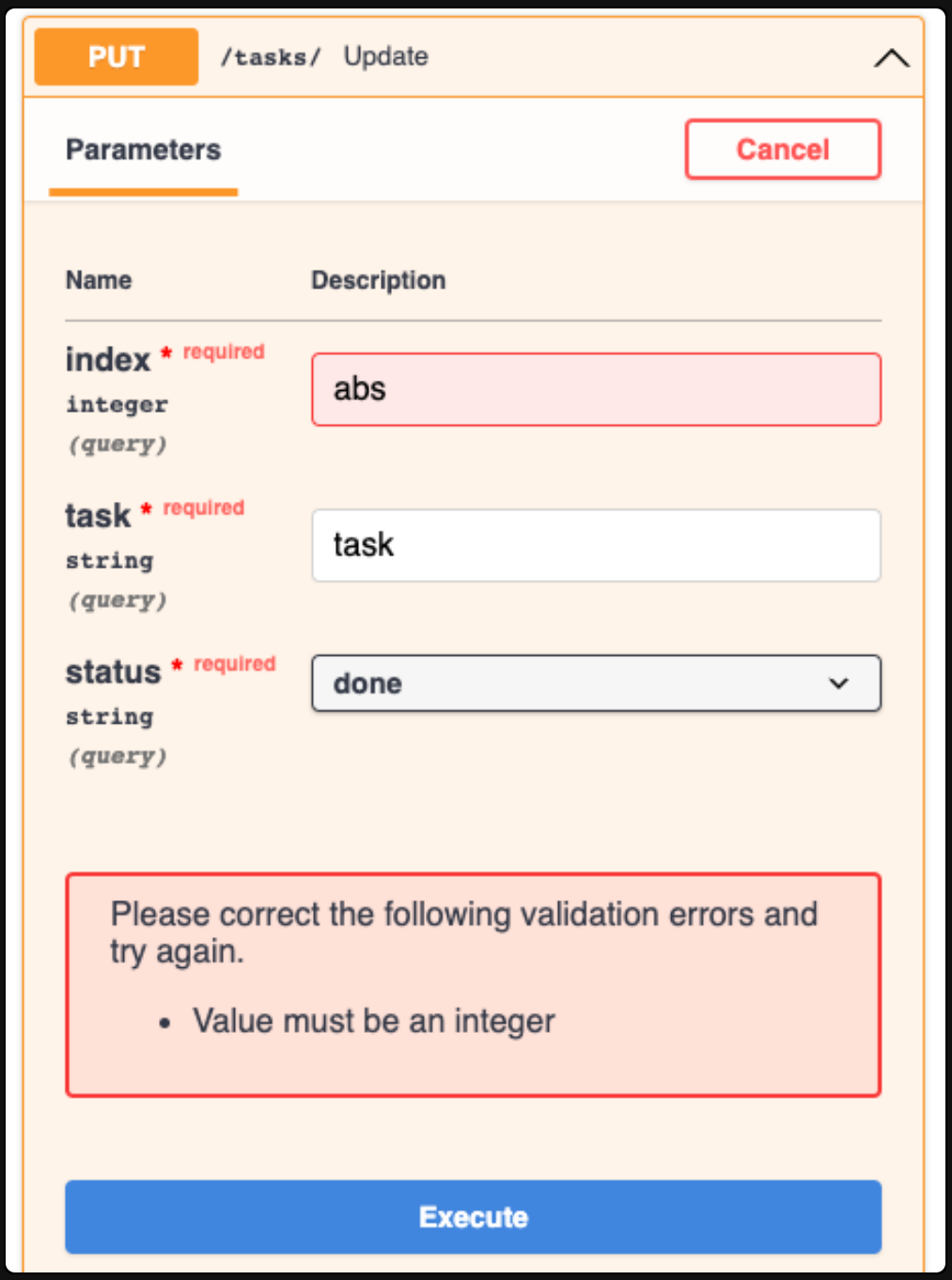
Presionas sobre Try it out, y colocas los datos solicitados (si lo solicita):



Y envías la petición; veras la respuesta:



También es importante notar que, al definir el tipo, existen validaciones aplicadas por defecto; en el caso anterior, colocamos que todos los argumentos deben ser de tipo entero para el índice y de tipo string para la tarea; así que, si por ejemplo es suministrado un tipo de dato incorrecto; por ejemplo, un texto en lugar de un entero, veremos un error como el siguiente:



Como tarea, haz varias pruebas de los métodos anteriores y evalúa la respuesta, es decir, la petición y el curl creado.

Con esto, tendrás una referencia exacta de cómo armar tus métodos para que tengan el comportamiento que esperas que tenga.

Crear una tarea (POST):

```
curl -X 'POST' \
  'http://127.0.0.1:8000/New%20Task' \
  -H 'accept: application/json' \
  -d ''
```

Request URL

http://127.0.0.1:8000/New%20Task

Obtener todas las tareas:

```
curl -X 'GET' \
  'http://127.0.0.1:8000/' \
  -H 'accept: application/json'
```

Request URL

http://127.0.0.1:8000/

Para actualizar una tarea:

```
curl -X 'PUT' \
  'http://127.0.0.1:8000/?task=Change%20Name%20Task&status=ready' \
  -H 'accept: application/json'
```

Request URL

http://127.0.0.1:8000/?task=Change%20Name%20Task&status=ready

Para eliminar una tarea:

```
curl -X 'DELETE' \
  'http://127.0.0.1:8000/?index=0' \
  -H 'accept: application/json'
```

Request URL

http://127.0.0.1:8000/?index=0

Todos los métodos definidos anteriormente utilizan parámetros vía GET:

http://127.0.0.1:8000/?index=0

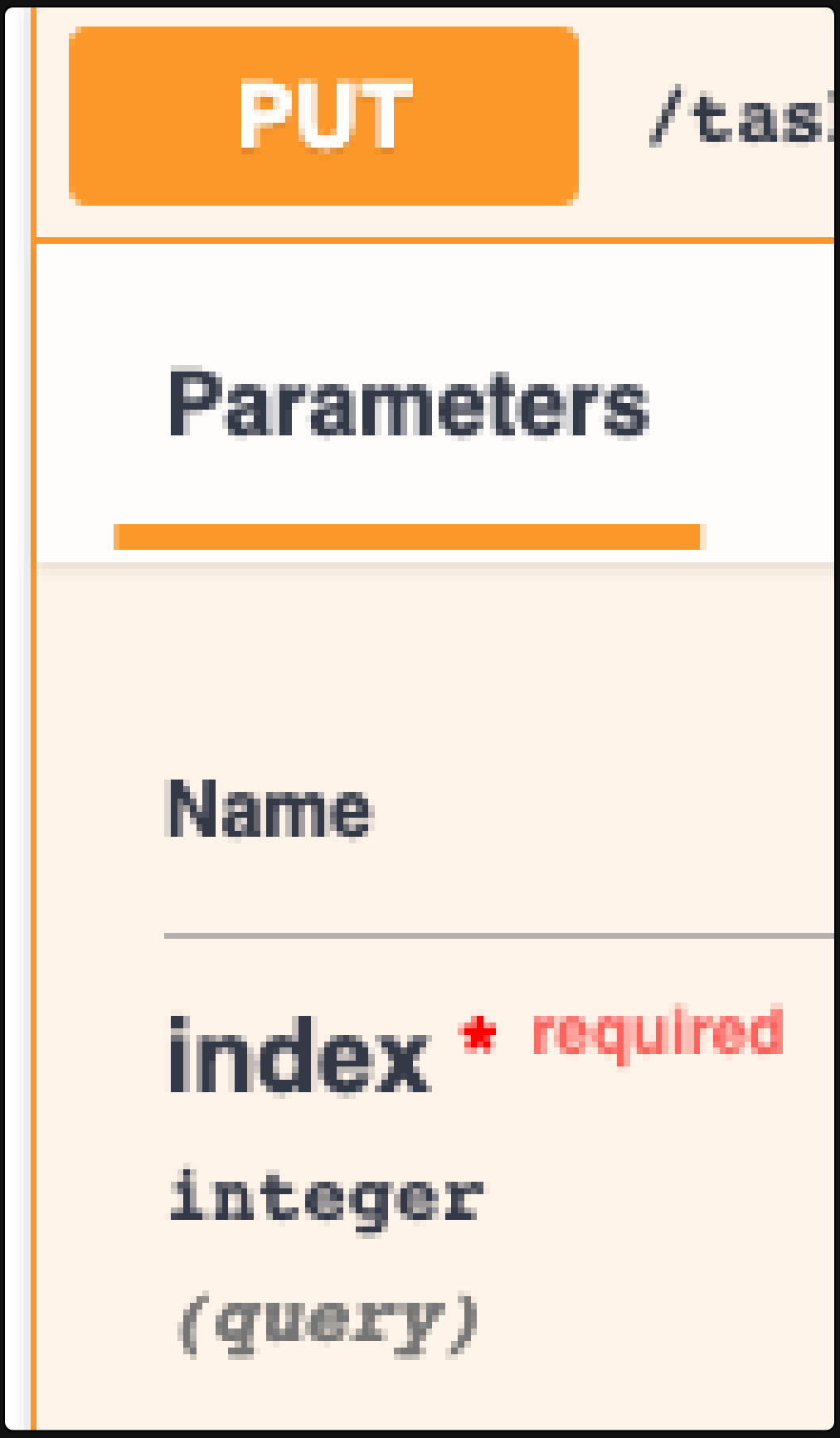
A excepción del método para crear una tarea; en el cual, el dato (tarea a crear) está definida directamente en la URI:

http://127.0.0.1:8000/New%20Task

Y esto es debido a que, en el caso de crear la tarea, el argumento se especifica a nivel del path:

```
@task_router.post("/{task}")
```

También, puedes ver especificado en el parámetro, el comportamiento de los datos solicitados al usuario, si es mediante la query string:



O formar parte del path en la URL como en el caso del método POST:

POST /tas

Parameters

Name

task * required

string

(path)

Y es justamente por la forma en la cual definimos el path para cada método.

Limitar los valores permitidos (datos predefinidos)

Muchas veces tenemos algunos campos con valores fijos, como indicar un tipo de usuario, el estado de una publicación, o en este caso, el estado de la tarea; que pueden ser, lista o pendiente; en Python, tenemos los tipos enumerados que podemos usar para tal fin; vamos a crear un archivo nuevo en el cual, colocaremos el estatus para las tareas:

models.py

```
from enum import Enum
class StatusType(str,Enum):
    DONE = "done"
    PENDING = "pending"
```

Luego, para usarlo desde el CRUD de tareas, lo único que debemos de hacer convertir cada posición de la lista en un tipo enumerado para recibir tanto la tarea como el estatus; por ejemplo:

```
task_list.append({
   'task':task,
   'status': StatusType.PENDING
})
```

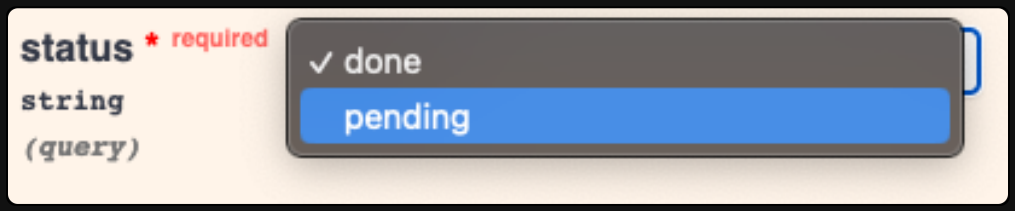
Teniendo esto claro, lo siguiente que hacemos es adaptar las funciones de add() y update():

```
@task_router.post("/{task}")
def add(task: str):
    task_list.append({
        "task" : task,
        "status" : StatusType.PENDING,
    })
    return { "tasks": task_list }
@task_router.put("/")
def update(index: int, task: str, status: StatusType):
    task_list[index] = {
        "task" : task,
        "status" : status,
    }
    return { "tasks": task_list }
```

Fijate que en la función de update(), se recibe como argumento el tipo enumerado:

```
update(index: int, task: str, status: StatusType)
```

Desde el apartado de documentación, veremos que cuando vamos a modificar una tarea, tenemos un campo de selección con los valores permitidos:



Parámetros con Query()

También tenemos acceso a validaciones más avanzadas a través de la clase Query() en la cual, podemos indicar desde valores mínimos y máximos, expresiones regulares y mucho más; por ejemplo, en la siguiente función podemos agregar validaciones en caso a límites de rango:

from fastapi import ***, Query

```
@app.get("/page")
def page(page: int = Query(1, ge=1, le=20), size: int = Query(5, ge=5, le=20)):
    return {"page": page, "size": size}
```

Con la clase Query() podemos indicar distintos argumentos para realizar validaciones numéricas; en el ejemplo anterior, usamos las opciones de:

- gt: Significa mayor que (Greater than).
- ge: Significa mayor o igual que (Greater than or equal to).
- lt: Significa menor que (Less than).
- le: Significa menor o igual que (Less than or equal to).

El primer parámetro corresponde con el valor por defecto que va a tener; por lo tanto, los parámetros serían opcionales.

En el ejemplo anterior, de indica que:

- El argumento page debe ser mayor o igual a 1 y menor o igual a 20.
- El argumento size debe ser mayor o igual a 5 y menor o igual a 20.

Podemos colocar otros tipos de opciones como expresiones regulares:

```
@app.get("/phone/{phone}") # +34 111 12-34-56
def phone(phone: str = Query(pattern=r"^(\(?\+[\d]{1,3}\)?)\s?([\d]{1,5})\s?([\d][\s\.-]?){6,7}$" )):
    return {"phone": phone}
```

Inclusive metadata:

```
@app.get("/page")
def page(page: int = Query(1, gt=0, title='Pagina a visualizar'), size: int = Query(10, le=100)):
    return {"page": page, "size": size}
```

Recuerda que para emplear la clase Query() los parámetros no tienen que estar definidos en el path ya que, para emplear la clase Query() significa que los argumentos van a pasarse por la query string, por lo tanto, si para el ejemplo anterior, colocaras el parámetro por el path:

```
@app.get("/phone/{phone}") # +34 111 12-34-56
def phone(phone: str = Query(pattern=r"^(\(?\+[\d]{1,3}\)?)\s?([\d]{1,5})\s?([\d][\s\.-]?){6,7}$" )):
    return {"phone": phone}
```

Al guardar verías un error como el siguiente:

Cannot use `Query` for path param 'phone'

Path con Path()

Al igual que ocurre con la clase Query() podemos definir los mismas opciones de validación, para indicar valores minimos o maximos al igual que expresiones regulares, con la diferencia de que, se debe de especificar el atributo como parte del path y la clase Path() no recibe valores por defecto:

```
from fastapi import ***, Path
***
@app.get("/page/{page}")
def page(page: int = Path(gt=0), size: int = Query(10, le=100)):
    return {"page": page, "size": size}
```

Importante definir el atributo como parte del path ya que, si no lo colocas:

```
from fastapi import ***, Path
***
@app.get("/page/")
def page(page: int = Path(gt=0), size: int = Query(10, le=100)):
    return {"page": page, "size": size}
```

Al enviar la petición al método anterior te indicará una excepción que indica el el argumento tipo Path (page en este ejemplo) no ha sido encontrado:

```
Code        Details
422
Error: Unprocessable Entity
Response body
Download
{
  "detail": [
    {
      "loc": [
        "path",
        "page"
      ],
      "msg": "field required",
      "type": "value_error.missing"
    }
  ]
}
```

Body

Todos los ejemplos realizados anteriormente hemos enviado los datos a la API mediante la URL, ya sea mediante el path o el query string, si has trabajado en otras tecnologías web del lado del servidor, debes de saber que esta no es la mejor manera que tenemos para pasar datos al servidor cuando queremos que estos datos se registren de manera persistente en la base de datos o similares, los datos que viajan en la URL, generalmente van acompañado de una petición de tipo GET, cuando queremos persistir estos datos o en general, cambiar el modelo de datos (entiéndase crear, actualizar o eliminar algo) se envían peticiones de tipo POST, PUT, PATH o DELETE y los datos van encapsulados en el cuerpo de la petición, es decir, en el body.

El cuerpo es la parte de la solicitud HTTP que contiene datos que van a ser procesados por el servidor en una API REST, generalmente se codifica en JSON y se usa para crear o editar objetos en una base de datos; aunque, todavía no vamos a persistir estos datos, ya viene siendo el momento de que vayamos a estructurar correctamente los datos en los métodos de la API según el propósito de la misma.

Así que, en pocas palabras, para poder enviar los datos en la API mediante el cuerpo y no por la URL, podemos usar la clase de Body() sobre el o los argumentos:

```
from fastapi import *** Body
@task_router.post("/{task}")
def add(task: str = Body()):
    task_list.append({
        'task':task,
        'status': StatusType.PENDING
    })
    print(task_list)
    return task_list
```

En la terminal veremos un error como el siguiente:

AssertionError: Cannot use `Body` for path param 'task'

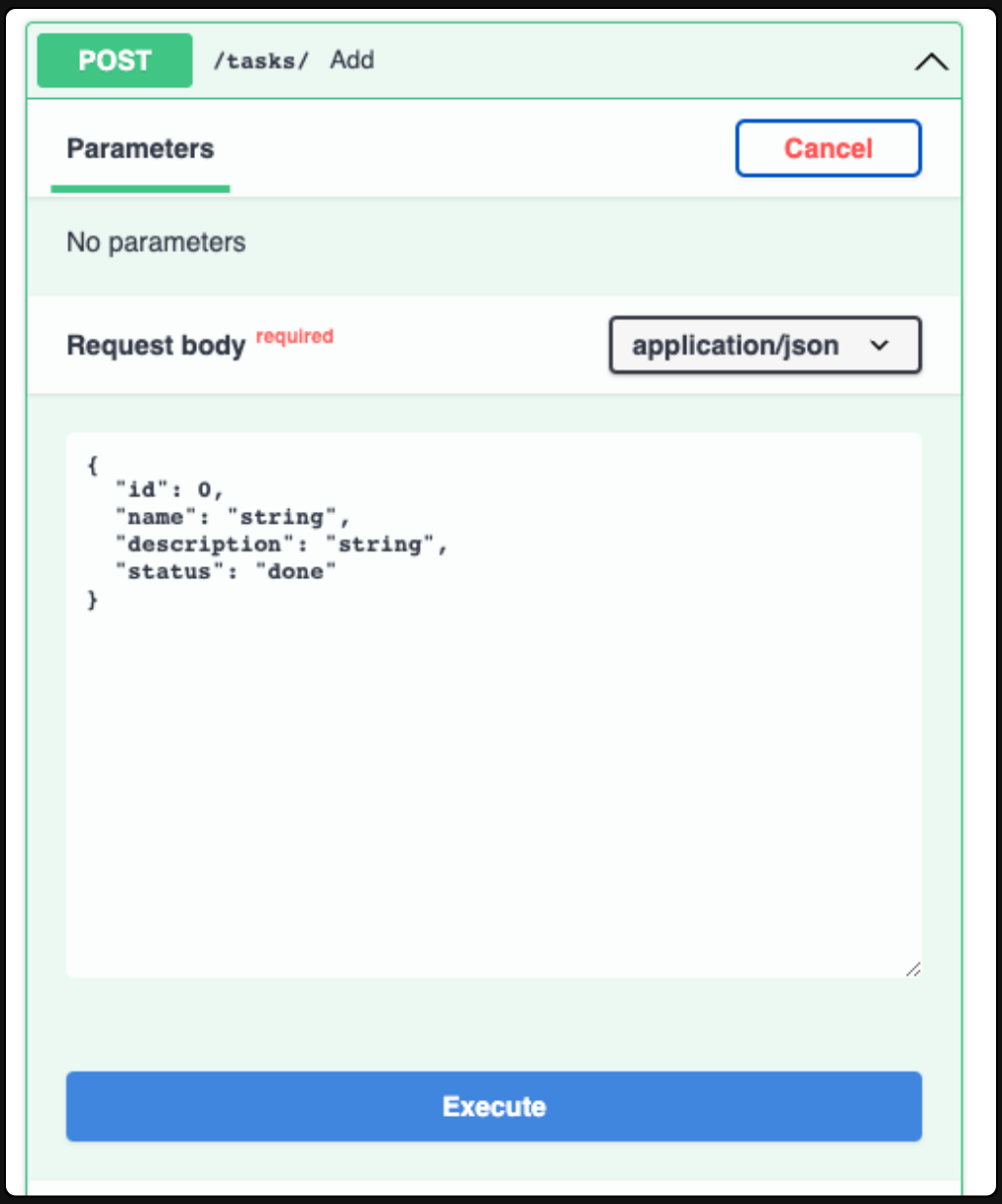
Ya que, ahora el argumento llamado task se establece en el body y no como parámetro de la URL; así que, se debe de remover del path:

```
@task_router.post("/")
def add(task: str = Body()):
    task_list.append({
        'task':task,
        'status': StatusType.PENDING
    })
    print(task_list)
    return task_list
```

Para el proceso de actualizar, hacemos los cambios correspondientes, en los campos que queremos que los datos vayan en el cuerpo de la petición, usamos la clase Body():

```
@task_router.put("/")
def update(index: int, task: str = Body(), status: StatusType = Body()):
    task_list[index] = {
        "task" : task,
        "status" : status,
```

Ahora, si vamos a la documentación, veremos que tenemos un campo de tipo TEXTAREA para colocar el contenido:



Es importante que al colocar el texto no elimines las comillas ya que, de esta manera no sería tratado como un texto y verías un error como el siguiente:

```
curl -X 'POST' \
  'http://localhost:8000/' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d 'Task Content'
Request URL
http://localhost:8000/
422
Error: Unprocessable Entity
Response body
Download
{
  "detail": [
    {
      "loc": [
        "body",
        0
      ],
      "msg": "Expecting value: line 1 column 1 (char 0)",
      "type": "value_error.jsondecode",
      "ctx": {
        "msg": "Expecting value",
        "doc": "Task Content",
        "pos": 0,
        "lineno": 1,
        "colno": 1
      }
    }
  ]
```

Si suministrar el argumento de manera correcta (con las comillas), verás una respuesta como:

```
curl -X 'POST' \
  'http://localhost:8000/' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '"Task Content"'
Request URL
```

http://localhost:8000/

Validación de cuerpos de solicitud utilizando modelos Pydantic

Pydantic es una biblioteca de Python para validación de datos; básicamente son una serie de clases en donde se define y aplica validaciones sobre los datos que se van a recibir en la petición del usuario, por lo tanto, se puede separar la presentación de estos datos, así como su estructura de los métodos que manejan la petición.

Como se mencionó anteriormente en el libro, Pydantic es utilizado por defecto como una de las bibliotecas más importantes de FastAPI, así que, no hay necesidad de instalar algún paquete adicional ya que, viene incorporado al instalar FastApi.

Un ejemplo de modelo es el siguiente:

models.py

```
from pydantic import BaseModel
from enum import Enum
class StatusType(str, Enum):
    READY = "ready"
    PENDING = "pending"
class Task(BaseModel):
    id: int
    name: str
    description: str
    status: StatusType
```

Es una clase típica en Python que hereda de BaseModel de la biblioteca de Pydantic; el cambio anterior debes de hacerlo en el archivo de models.py; ahora, podemos simplificar los métodos de la API de la siguiente manera:

task.py

```
@task_router.post("/")
def add(task: Task):
    # task_list.append({
    #     "task" : task.name,
    #     "status" : task.status,
    # })
    task_list.append(task)
    return { "tasks": task_list }
@task_router.put("/")
def update(index: int, task: Task):
    task_list[index] = {
        "task" : task.name,
        "status" : task.status,
    }
    return { "tasks": task_list }
```

Como puedes apreciar, ya no hay necesidad de recibir múltiples argumentos, solamente uno de ellos, una instancia de la clase modelo definida antes; importante notar que, esta instancia es suministrada directamente en el body, por lo tanto, no debe de estar definida en el path de la siguiente manera:

```
@task_router.post("/{task}")
def add(task: Task):
  ***
```

O veras un error como el siguiente:

```
Path params must be of one of the supported types
```

Validaciones

En las clases de los modelos es posible aplicar validaciones; para ello, se utiliza el decorador de validación (indicando el campo a validar) sobre una función que recibe el modelo y el valor; desde esta función se pueden definir distintas reglas y devolver una excepción en caso de que el valor no satisfaga las reglas; veamos un ejemplo:

models.py

```python
from pydantic import BaseModel,  ValidationError, field_validator
class Task(BaseModel):
    id: int
    name: str
    description: str
    status: StatusType

    @field_validator('id')
    def greater_than_zero(cls, v):
        if v <=0 :
            raise ValueError('must be greater than zero')
        return v
```

En el ejemplo anterior, se aplica una regla de validación sobre el campo id, la cual consiste en que el valor debe ser mayor a cero; el valor retornado a la función corresponde con el valor que va a recibir los métodos de la API; usualmente vamos a querer retornar el valor recibido por el usuario como se muestra en el código anterior.

Desde la documentación, si ingresamos un valor menor o igual a cero, veremos un error como el siguiente:

```
422
Error: Unprocessable Entity
Response body
Download
{
  "detail": [
    {
      "loc": [
        "body",
        "id"
      ],
      "msg": "must be greater than zero",
      "type": "value_error"
    },
    {
      "loc": [
        "body",
        "name"
      ],
      "msg": "must be alphanumeric",
      "type": "assertion_error"
    }
  ]
}
```

Es decir, la excepción que definimos anteriormente; puedes crear más métodos sobre el mismo campo:

```
class Task(BaseModel):
    ***
    @field_validator('id')
    def id_greater_than_zero(cls, v):
        if v <=0 :
            raise ValueError('must be greater than zero')
        return v

    @field_validator('id')
    def id_less_than_a_thousand(cls, v):
        if v >1000 :
            raise ValueError('must be less less than a thousand')
        return v
```

U otros campos, por ejemplo, el de nombre:

```
class Task(BaseModel):
    ***
    @field_validator('name')
    def id_name_alphanumeric(cls, v):
        assert v.replace(" ", "").isalnum(), 'must be alphanumeric'
        return v
```

En esta validación, se indica que el nombre solamente puede tener caracteres alfanuméricos y espacios en blanco; se usa la función de replace() para remover los espacios en blanco y luego usar la función de isalnum() que retorna true si el string a evaluar está compuesto por caracteres alfanuméricos.

Puedes obtener más información en:

https://docs.pydantic.dev/latest/usage/validators/

Anidar clases

Podemos anidar fácilmente clases para representar objetos más completos, por ejemplo, podemos indicar que cada tarea tiene una categoría asociada de la siguiente manera:

```
class Category(BaseModel):
    id: int
    name: str
class Task(BaseModel):
    ***
    category: Category
```

O por ejemplo, un usuario dueño de la tarea y la categoría:

```python
class User(BaseModel):
    id: int
    name: str
    surname: str
    email: str

class Category(BaseModel):
    id: int
    name: str
class Task(BaseModel):
    ***
    user: User
    category: Category
```

Al crear o actualizar una tarea, veremos que las relaciones ahora forman parte de la petición:

```json
{
  "id": 0,
  "name": "string",
  "description": "string",
  "status": "done",
  "category": {
    "id": 0,
    "name": "string"
  },
  "user": {
    "id": 0,
    "name": "string",
    "surname": "string",
    "email": "string"
  }
}
```

Herencia

Para mantener más modulares estas clases de validación, podemos usar las primitivas básicas como la herencia; actualmente, tenemos 3 entidades, tareas, usuarios y categorías, cada una puede manejar un ID con las validaciones que definimos antes; pensando en esto, crearemos una clase base como la siguiente:

models.py

```
***
class MyBaseModel(BaseModel):
    id: int
    @field_validator('id')
    def greater_than_zero(cls, v):
        if v <=0 :
            raise ValueError('must be greater than zero')
        return v

    @field_validator('id')
    def less_than_a_thousand(cls, v):
        if v >1000 :
            raise ValueError('must be less less than a thousand')
        return v
```

Y ahora, heredamos desde esta nueva clase para cada una de las entidades definidas:

models.py

```
***
class User(MyBaseModel):
    id: int
    name: str
    surname: str
    email: str

class Category(MyBaseModel):
    id: int
    name: str
    # validaciones de nombre e email TODO
class Task(MyBaseModel):
    name: str
    description: str
    status: StatusType
    user: User
    category: Category

    @field_validator('name')
    def name_alphanumeric(cls, v):
        assert v.replace(" ", "").isalnum(), 'must be alphanumeric'
        return v
```

Validación por campos

Al igual que vimos con el uso de las clases de Query() o Path(), podemos aplicar las mismas validaciones desde las clases modelos; por ejemplo:

models.py

```
from pydantic import *** Field
class StatusType(str, Enum):
    READY = "ready"
    PENDING = "pending"
class MyBaseModel(BaseModel):
    id: int = Field(ge=1, le=100)
    ***

class User(MyBaseModel):
    name: str = Field(min_length=3)
    ***
```

Estos valores son netamente de pruebas y no tienen un uso real para la aplicación que estamos construyendo, por lo tanto, puedes prescindir de estas validaciones o personalizarlas a gusto.

Valores opcionales y valores por defecto

Otro factor importante es que, muchas veces tenemos parámetros que son opcionales, por ejemplo, para la aplicación de tareas, la descripción puede ser opcional; así que, pensando en eso, tenemos:

models.py

```
from typing import Optional
***
class Task(MyBaseModel):
    ***
    description: Optional[str]
```

O si tiene validaciones:

```
from typing import Optional
***
class Task(MyBaseModel):
    ***
    description: Optional[str] = Field("No description", min_length=3)
```

O si quieres colocar un valor por defecto cuando el mismo no es especificado al momento de hacer la petición:

```
from typing import Optional
***
class Task(MyBaseModel):
    ***
    description: Optional[str] = Field("No description", min_length=3)
```

Si haces una petición sin enviar la descripción, verás que toma el valor por defecto:

```
curl -X 'POST' \
  'http://localhost:8000/tasks/' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "id": 100,
  "name": "string",
  "status": "done",
  "category": {
    "id": 100,
    "name": "string"
  },
  "user": {
    "id": 100,
    "name": "string",
    "surname": "string",
    "email": "user@example.com",
    "website": "string"
  }
}'
{
  "tasks": [
    {
      "id": 100,
      "name": "string",
      "description": "No description",
      "status": "done",
      "category": {
        "id": 100,
        "name": "string"
      },
      "user": {
        "id": 100,
        "name": "string",
        "surname": "string",
        "email": "user@example.com",
        "website": "string"
      }
    }
  ]
}
```

Caso contrario, si colocas una descripción válida, veras el valor establecido:

```
curl -X 'POST' \
  'http://localhost:8000/tasks/' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "id": 100,
  "name": "string",
  "description": "string",
  ***
}'
200
Response body
Download
{
  "tasks": [
    {
      "id": 100,
      "name": "string",
      "description": "string",
      ***
  ]
}
```

Si colocas una descripción invalida, en este ejemplo, por la longitud, veras la excepción:

```
curl -X 'POST' \
  'http://localhost:8000/tasks/' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "id": 100,
  "name": "string",
  "description": "bu",
  ***
}'
422
Error: Unprocessable Entity
Response body
Download
{
  "detail": [
    {
      "loc": [
        "body",
        "description"
      ],
      "msg": "ensure this value has at least 5 characters",
      "type": "value_error.any_str.min_length",
      "ctx": {
        "limit_value": 5
      }
    }
  ]
}
}'
```

También puedes especificar el valor por defecto, que sea nulo:

```
from typing import Optional
***
class Task(MyBaseModel):
    ***
    description: Optional[str] = Field(None, min_length=3)
```

Tipos de campos

Muchas veces tenemos que colocar validaciones para formatos específicos como IPs, JSON o email; para estos casos, ya existen clases que podemos aplicar directamente sobre las propiedades de la clase:

models.py

```
from pydantic import *** EmailStr, HttpUrl
***
class User(MyBaseModel):
    ***
    email: EmailStr
    website: HttpUrl
```

Si vemos un error como el siguiente:

email-validator is not installed, run `pip install pydantic[email]

Debes de instalar el paquete adicional:

```
$ pip install pydantic[email]
```

Si te da una excepción:

```
no matches found: pydantic[email]
```

Prueba colocando unas comillas:

```
$ pip install 'pydantic[email]'
```

Ya con esto, tenemos acceso a validaciones adicionales y en caso de no pasar las validaciones, tendremos un error como el siguiente:

```
{
  "detail": [
    {
      ***
    {
      "loc": [
        "body",
        "user",
        "email"
      ],
      "msg": "value is not a valid email address",
      "type": "value_error.email"
    },
    {
      "loc": [
        "body",
        "user",
        "website"
      ],
      "msg": "invalid or missing URL scheme",
      "type": "value_error.url.scheme"
    },
}
```

Tienes la lista completa de validaciones que puedes usar en:

https://docs.pydantic.dev/latest/usage/types/#pydantic-types

Usar una lista como argumento

Otra característica interesante es la de poder establecer una lista de valores u objetos como parte de la clase modelo; para ello:

models.py

```
from typing import List
***
class Task(MyBaseModel):
    name: str
    description: Optional[str] = Field("No description", min_length=3)
    status: StatusType
    user: User
    category: Category
    tags: List[str] = []
```

Con esto, podemos pasar una lista de valores:

```
{
  ***
  "tags": ["tag 1","tag 2"]
}
```

Si quieres que los valores sean únicos y no se puedan repetir, puedes usar un set():

```
class Task(MyBaseModel):
    name: str
    description: Optional[str] = Field("No description", min_length=3)
    status: StatusType
    user: User
    category: Category
    # tags: List[str] = []
    tags: set[str] = set()
Si le pasamos los siguientes datos:
{
  ***
  "tags": ["tag 1","tag 2","tag 3","tag 1"]
}
```

Obtendremos el siguiente listado con valores únicos si la lista fue definida con la función de set():

```
***
"tags": [ "tag 1", "tag 3", "tag 2" ]
Si la lista fue definida mediante la clase List:
***
"tags": ["tag 1","tag 2","tag 3","tag 1"]
```

FastAPI documentación automática

Como hemos visto en los anteriores ejemplos, FastAPI genera la documentación de manera automática para las rutas que tengamos definidas en la aplicación, desde estas rutas, podemos realizar peticiones, indicando parámetros a las rutas, body y mucho más; pero, actualmente existen dos tipos de documentaciones que podemos usar:

Swagger

Esta es la documentación que hemos usado hasta este momento mediante:

http://127.0.0.1:8000/docs

Y proporciona un entorno interactivo para probar nuestra API, no importa el tipo de petición que se haya definido que se puede probar desde aquí, así que, tenemos tanto la definición de las distintas rutas de la aplicación como también la opción de poder probar las rutas.

ReDoc

Al igual que la anterior, esta versión ofrece un entorno para poder ver la definición de las distintas rutas al igual que probar la API; aunque, se podría decir que ofrece una interfaz más detallada y de fácil acceso para cada uno de los métodos de la API; puedes probar esta versión de la documentación desde:

http://127.0.0.1:8000/redoc

La diferencia principal entre ambas documentaciones es que en la de redoc no tenemos el botón de "Try it out" para probar los métodos.

Conclusiones

Cada una de los apartados que vimos antes representan las distintas formas que existen en FastApi para hacer tanto el ruteo, como pasar los datos a la Api; puedes mezclar estas soluciones y adaptar la que más se ajuste al método o métodos de tu Api; vimos distintos esquemas para definir los

argumentos, ya sea mediante el path, query string, el body, validaciones, valores por defecto, opcionales y muchos otros esquemas.

Código fuente del capítulo:

https://github.com/libredesarrollo/curso-libro-fast-api-crud-task-1/releases/tag/v0.1